

BÁO CÁO DỰ ÁN MÔN HỌC

HỆ THỐNG ĐẤU GIÁ TRỰC TUYẾN

Danh sách thành viên:
Nhóm trưởng – Lê Nguyễn Quốc Bình
Nguyễn Đức Anh
Lê Gia Bảo
Đặng Tuấn Đức

1. Mục tiêu và phạm vi

Mục tiêu: Xây dựng hệ thống đấu giá trực tuyến client-server dùng JavaFX và socket JSON. Ứng dụng cho phép tạo phiên, theo dõi, đặt giá realtime, tự động kết thúc, xác định winner và giữ dữ liệu đúng khi nhiều bid đến đồng thời. Trọng tâm là luật bid rõ ràng, cập nhật tức thời và GUI đủ cho người mua, người bán, admin.

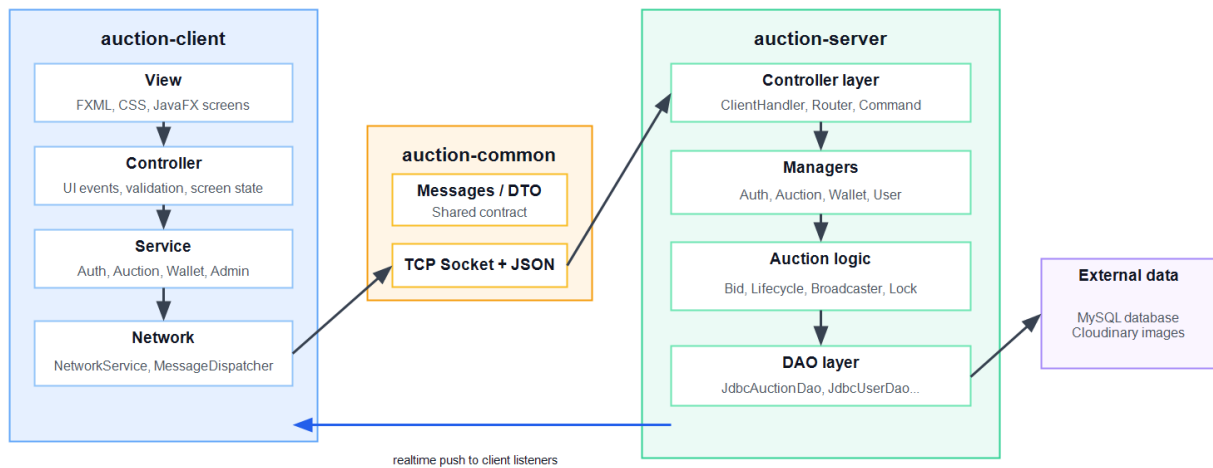
Phạm vi:

- Đăng ký, đăng nhập username/password; phân quyền User/Admin.
- Tạo, cập nhật, hủy, đóng phiên đấu giá.
- Đặt giá, validate bid, cập nhật giá hiện tại và người dẫn đầu.
- JavaFX GUI: login/register, auction list/detail, create/manage auction, wallet, activity, admin panel.
- Nâng cao: realtime update, anti-sniping, chart lịch sử bid, xử lý bid đồng thời.

2. Kiến trúc hệ thống và MVC

Sơ đồ kiến trúc tổng thể:

System Architecture - Online Auction App



Observer: observeAuction registers server subscribers; addMessageListener handles pushed messages on the client.

- **auction-client:** JavaFX client gồm View, Controller, Service và Network. View là FXML/CSS; Controller xử lý sự kiện UI; Service tạo request; NetworkService gửi/nhận message.
- **auction-common:** chứa Message, DTO, enum và JSON util. Module này là hợp đồng chung để client và server hiểu cùng một định dạng dữ liệu.
- **auction-server:** nhận socket qua ClientHandler, điều hướng bằng MessageRouter/MessageCommand, xử lý nghiệp vụ trong Manager/Auction logic, sau đó lưu dữ liệu qua DAO.
- **External data:** MySQL lưu user, auction, bid, wallet (database được host online trên Aiven), Cloudinary lưu ảnh sản phẩm.

MVC phía client:

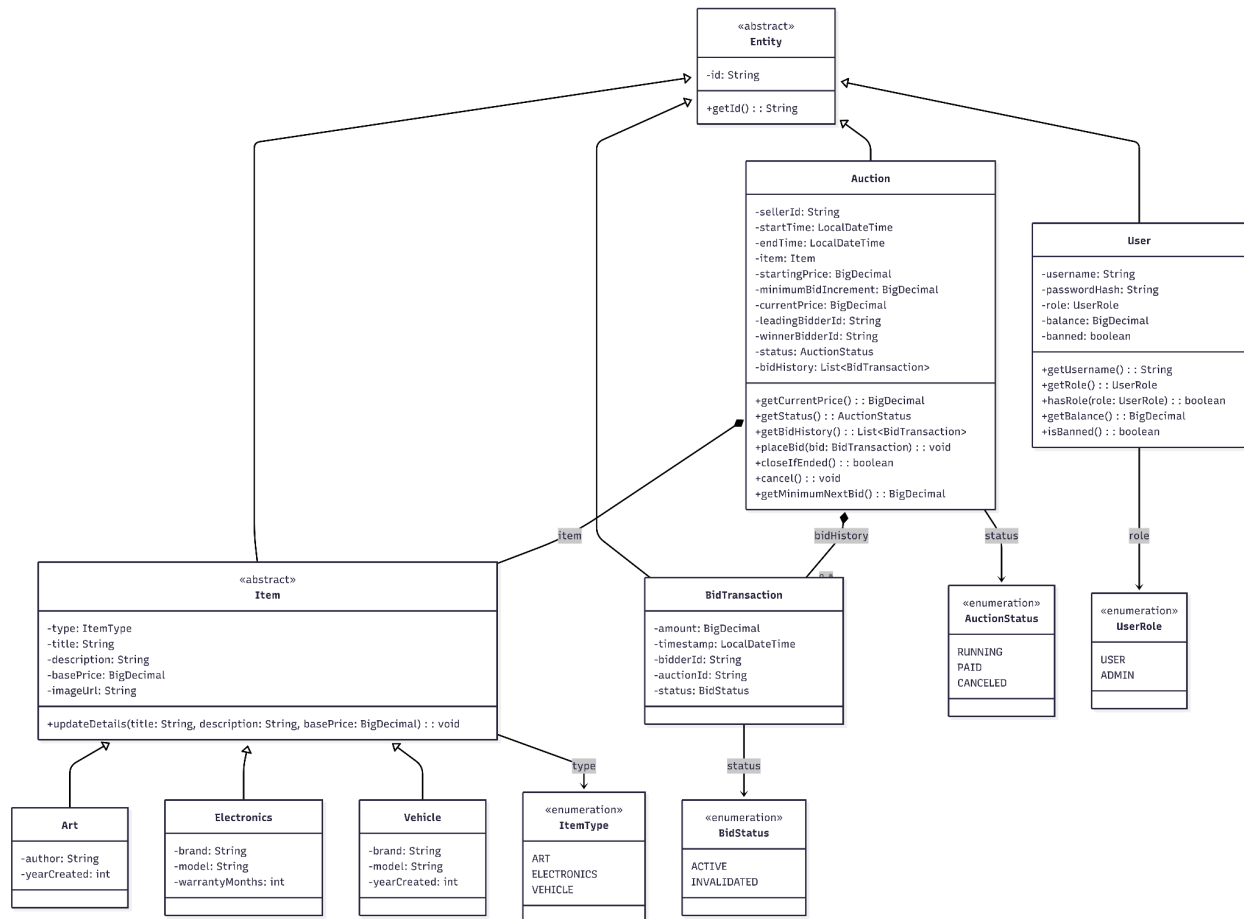
- **View:** FXML như AuctionListMenu.fxml, AuctionItemMenu.fxml mô tả giao diện, input, button, table, chart; không chứa luật nghiệp vụ.
- **Controller:** LoginMenuController, AuctionListMenuController, AuctionItemMenuController nhận sự kiện, đọc dữ liệu nhập, gọi service và cập nhật giao diện.
- **Model/Service:** AuctionService, AuthService, WalletService tạo request message và nhận response qua NetworkService.

MVC phía server:

- **Controller layer:** ClientHandler nhận message từ socket; MessageRouter dựa vào MessageType để gọi MessageCommand.
- **Model/Business layer:** Auction, BidTransaction, Item, User và manager chứa trạng thái, rule, nghiệp vụ.
- **DAO layer:** JdbcAuctionDao, JdbcUserDao, JdbcBalanceDao tách SQL khỏi business logic.

3. OOP và thiết kế lớp

Sơ đồ lớp models:



Bốn nguyên tắc OOP:

- **Encapsulation:** Auction, User, Item, BidTransaction giữ field private/protected và đổi dữ liệu qua method. Ví dụ Auction.placeBid() kiểm tra trạng thái, giá tối thiểu, thời hạn rồi mới cập nhật currentPrice, leadingBidderId, bidHistory.
- **Inheritance:** Entity là lớp gốc có id. User kế thừa Entity. Item là abstract class kế thừa Entity; Art, Electronics, Vehicle kế thừa Item để dùng chung name, description, price và thêm thuộc tính riêng.

- **Polymorphism:** Code xử lý sản phẩm bằng kiểu Item chung nhưng runtime có thể là Art, Electronics hoặc Vehicle.
- **Abstraction:** Item, User và DAO interface như AuctionDao, UserDao chỉ công khai hành vi cần dùng. Manager làm việc qua interface; JdbcAuctionDao/JdbcUserDao chứa chi tiết JDBC.

4. Design Patterns

Pattern	Áp dụng trong hệ thống
Singleton	AuctionManager, AuthManager, WalletManager, DatabaseConnection cung cấp một instance dùng chung duy nhất
Factory Method (để tạo item)	ItemFactory là abstract factory; ArtFactory, ElectronicsFactory, VehicleFactory tạo đúng subclass Item theo loại sản phẩm.
Observer (cho luồng realtime update)	AuctionBroadcaster lưu các ClientHandler đang theo dõi auction (subscribers). Khi auction thay đổi, server notify các subscribers; phía client dùng listener để cập nhật UI realtime.
Command (để server xử lý message từ client)	MessageRouter map MessageType sang MessageCommand. Mỗi command xử lý một loại request, giúp mở rộng message dễ dàng
DAO	Jdbc...Dao tách câu lệnh SQL và logic truy cập database khỏi nghiệp vụ.

5. Chức năng và giải pháp chính

5.1. User, product và wallet

Register/Login dùng username/password, password hash bằng BCrypt. User tạo và quản lý listing; admin quản lý user. Wallet chia available balance và pending balance để khóa tiền đã cam kết, tránh dùng cùng một khoản tiền cho nhiều bid.

5.2. Bid validation và xử lý lỗi

- **Server validate bid:** auction phải RUNNING và chưa hết hạn; bid phải dương và $\geq \text{currentPrice} + \text{minimumBidIncrement}$; seller không được bid sản phẩm của mình; user không bị ban; ví phải đủ tiền. Request không hợp lệ được trả về client để UI hiển thị toast.
- **Custom exceptions để phân loại lỗi:** ValidationException cho input không hợp lệ, InvalidAuctionStateException cho auction đã đóng hoặc không RUNNING, AuthenticationException/AuthorizationException cho lỗi đăng nhập và phân quyền, InsufficientFundsException cho ví không đủ tiền, DatabaseException cho lỗi MySQL, NetworkException cho lỗi kết nối client-server.

5.3. Concurrent bidding

AuctionBid.submitBid dùng AuctionMutationExecutor với **ReentrantLock** để đảm bảo mỗi auction chỉ có một luồng được thay đổi tại một thời điểm:

1. Khóa mutation của auction.
2. Tạo Auction.snapshotCopy().
3. Validate và cập nhật snapshot bằng Auction.placeBid().
4. Persist bid và tiền bị khóa vào Database.
5. Persist thành công mới Auction.applySnapshot() vào auction thật.

LỢI ÍCH:

- **Lock để thread-safe:** Khi nhiều người bid cùng lúc, các request phải xếp hàng qua cùng một lock. Bid sau luôn đọc trạng thái mới nhất sau bid trước, nên không có trường hợp hai luồng cùng thấy một currentPrice cũ rồi cùng thắng. Cách này tránh lost update, price rollback và duplicate winner.
- **Snapshot copy để tránh lỗi database làm sai state:** Hệ thống không sửa trực tiếp auction thật ngay từ đầu, mà validate và cập nhật trên bản sao. Nếu thao tác trong database bị lỗi, auction gốc vẫn chưa đổi. Chỉ khi persist thành công, Auction.applySnapshot() mới ghi currentPrice, leadingBidderId, bidHistory và endTime vào auction thật.

5.4. Auction ending, anti-sniping và realtime

- **Auction ending:** AuctionLifecycle dùng ScheduledExecutorService kiểm tra mỗi giây. Khi hết hạn, Auction.closeIfEnded() đổi status sang PAID/CANCELED.

- **Anti-sniping:** Nếu bid trong 30 giây cuối, tăng endTime thêm 60 giây.
- **Realtime update được triển khai theo Observer Pattern:**
 - *Phía client:* Khi mở màn hình chi tiết, Controller gửi observe request lên server. Controller cũng đăng ký listener cho PRICE_UPDATE, AUCTION_UPDATED và AUCTION_ENDED. Khi nhận message được push về, listener cập nhật giá hiện tại, người dẫn đầu, thời gian còn lại và LineChart.
 - *Phía server:* AuctionManager nhận request observe/unobserve. AuctionBroadcaster đóng vai trò subject/publisher, lưu danh sách ClientHandler subscribers theo auctionId. Khi update auction, AuctionBroadcaster notify đúng nhóm subscribers đang theo dõi auction đó, tránh gửi update cho các client không liên quan.

6. Công cụ và kiểm thử

- **Maven:** quản lý dependencies, build đa module.
- **JUnit 5:** test bid, invalid bid, auction query, cancel bid, auth, notification, watchlist.
- **GitHub Actions:** .github/workflows/maven.yml chạy mvn clean verify, upload JAR, tạo release artifact.
- **SLF4J + Logback:** logging.

7. Phân công

Thành viên	Công việc đảm nhiệm
Quốc Bình	Networking, realtime update, auction item menu, auth, CI/CD, database connection pool / DAO, Cloudinary image
Đức Anh	Dashboard menu, wallet balance / pending balance, anti-snipe, custom exceptions, viết README
Gia Bảo	Login/register menu, model design, factory method để tạo item, unit test, checkstyle.
Tuấn Đức	My activities / my listings menu, CSS styling cho UI, realtime bid chart, watchlist, notification, admin panel UI.